Public - عام

MINISTRY OF EDUCATION
IMAM ABDULRAHMAN BIN
FAISAL UNIVERSITY
COLLEGE OF COMPUTER SCIENCE
& INFORMATION TECHNOLOGY

وزارة التعليم
جامعة الإمام
عبد الرحمن بن فيصل
كلية علوم الحاسب
وتقنية المعلومات

جامعة الإمام عبد الرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY

ARTI 502
Deep Learning: 2st Assignment
Training-Testing CNN with CIFAR Dataset using Pytorch
Year 2026 Term 1

Group # 1
Group Members:

#	Member Name	Member Academic ID
1	Fedaa Alsabea	2230003675
2	Hessa Al-Turki	2230007134
3	Maha Al-Mansoori	2230002711
4	Sara Al-Idrissi	2230004327
5	Rimas Alajmi	2230002688
6	Fatima Al- Shihab	2230002074

Date: 8 November 2026

Public - عام

Public - عام

List of Figures

Figure	Caption	Page
Figure 1	CIFAR-10 Dataset Download and Loading Code	4
Figure 2	Successful CIFAR-10 Dataset Loading Output	5
Figure 3	Training and Testing DataLoaders Implementation and Output	5
Figure 4	Baseline CNN Architecture Implementation and Model Summary	6
Figure 5	Training Function for the Baseline CNN	7
Figure 6	Testing Function for the Baseline CNN	7
Figure 7	Image Tensor Dimensions and Output	8
Figure 8	CIFAR-10 Dataset Visualization Code	9
Figure 9	Visualization of 10 CIFAR-10 Sample Images with Class Labels	9
Figure 10	Implementation of Six Data Augmentation Configurations and Image Transformation Functions for CIFAR-10	10
Figure 11	Original CIFAR-10 Frog Image Compared with Individual and Combined Augmentation Techniques	11
Figure 12	Baseline CNN Training and Testing Results over Five Epochs	12
Figure 13	CNN Training and Evaluation Code	13
Figure 14	Data Augmentation Performance Comparison	13
Figure 15	Training Configuration for CNN Architecture Experiments	14
Figure 16	Architecture A CNN Model Implementation	15
Figure 17	Architecture A Training and Testing Results	15
Figure 18	Architecture B CNN Model Implementation	16
Figure 19	Architecture B Training and Testing Results	16
Figure 20	Architecture C CNN Model Implementation	17
Figure 21	Architecture C Training and Testing Results	17
Figure 22	Training Loss Comparison Across CNN Architectures	18
Figure 23	Test Loss Comparison Across CNN Architectures	18
Figure 24	Test Accuracy Comparison Across CNN Architectures	18
Figure 25	Hyperparameter Tuning Function Implementation	19
Figure 26	Learning Rate Experiment Implementation	20
Figure 27	CNN Training and Evaluation Results for Different Learning Rates	20
Figure 28	Batch Size Experiment Implementation	21
Figure 29	CNN Training and Evaluation Results for Different Batch Sizes	21
Figure 30	Epoch Experiment Implementation	22
Figure 31	CNN Training and Evaluation Results for Different Numbers of Epochs	22
Figure 32	Summary of Hyperparameter Tuning Results	23
Figure 33	Final CNN Model Configuration and Selected Training Parameters	24
Figure 34	Final Model Training and Testing Results over 10 Epochs	25

Public - عام

Public - عام

Overview

In this assignment, we will have you train some convolutional neural networks and will use a small dataset (CIFAR).

Objective

To implement two core building blocks to deep learning:

- Convolution layers
- Maxpooling layers.

Public - عام

Public - عام

Tasks (with PyTorch)

Task 1: Getting the CIFAR Dataset

1.1 Dataset Download and Loading

The CIFAR-10 dataset was downloaded and loaded using PyTorch's torchvision.datasets.CIFAR10. The dataset contains 60,000 RGB images divided into 10 classes, with 50,000 images for training and 10,000 images for testing.

The ToTensor() transformation was applied to convert the images into PyTorch tensors. The dataset path was configured using DATA_PATH, and the training and testing datasets were loaded separately.

Additionally, DataLoaders were created with a batch size of 64 to organize the images into smaller batches. Shuffling was enabled for training and disabled for testing. This resulted in 782 training batches and 157 testing batches.

[2] ✓ 31m

```
# Set the dataset path
DATA_PATH = "./data"

# Convert images into tensors
transform = transforms.ToTensor()

# Download the training dataset
train_dataset = torchvision.datasets.CIFAR10(
    root=DATA_PATH,
    train=True,
    download=True,
    transform=transform
)

# Download the testing dataset
test_dataset = torchvision.datasets.CIFAR10(
    root=DATA_PATH,
    train=False,
    download=True,
    transform=transform
)

print("Training images:", len(train_dataset))
print("Testing images:", len(test_dataset))
```

Figure 1: CIFAR-10 Dataset Download and Loading Code

Public - عام

Public - عام


```
100%|██████████| 170M/170M [31:06<00:00, 91.4kB/s]
Training images: 50000
Testing images: 10000
```

Figure 2: Successful CIFAR-10 Dataset Loading Output

[3]
✓ 0s

```python
from torch.utils.data import DataLoader

# Create training and testing DataLoaders
train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=64,
    shuffle=False
)

print("Training batches:", len(train_loader))
print("Testing batches:", len(test_loader))
```

```
Training batches: 782
Testing batches: 157
```

Figure 3: Training and Testing DataLoaders Implementation and Output

Public - عام

Public - عام

Task 2:
Defining the Network
2.1 CNN Architecture:

Our CNN model is designed for the CIFAR-10 images, which have three color channels and a size of 32 × 32 pixels. It uses two convolutional layers to extract image features. The first layer has 32 filters, and the second has 64 filters. After each convolution, we apply ReLU and max-pooling. The resulting features are flattened and passed through a fully connected layer with 128 units. The final layer has 10 outputs, one for each CIFAR-10 class.

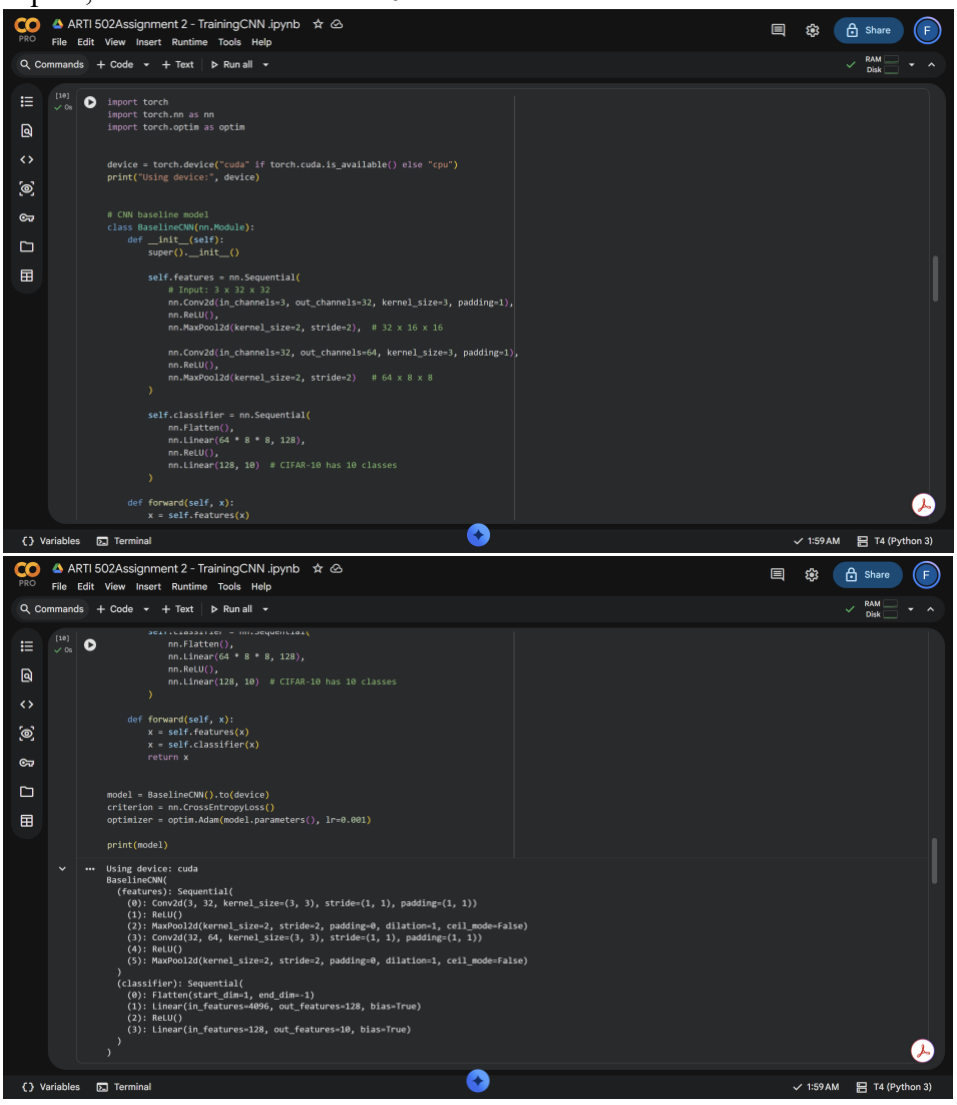


Figure 4: Baseline CNN Architecture Implementation and Model Summary

2.2 Training Function:

The training function trains the CNN using the training images in batches. For each batch, it makes predictions and calculates the loss. The optimizer then updates the model's weights to reduce the loss. At the end of each epoch, the function returns the average training loss.

Public - عام

Public - عام

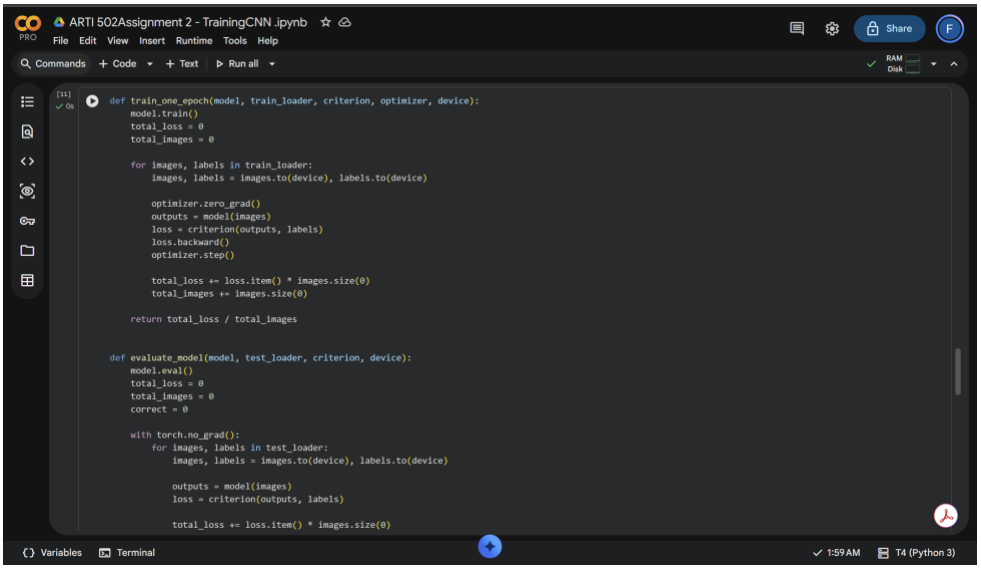


Figure 5: Training function for the baseline CNN

2.3 Testing Function:

The testing function checks how well the trained CNN performs on the test images. It calculates the test loss and counts how many images the model classifies correctly. The function returns the average test loss and the test accuracy.

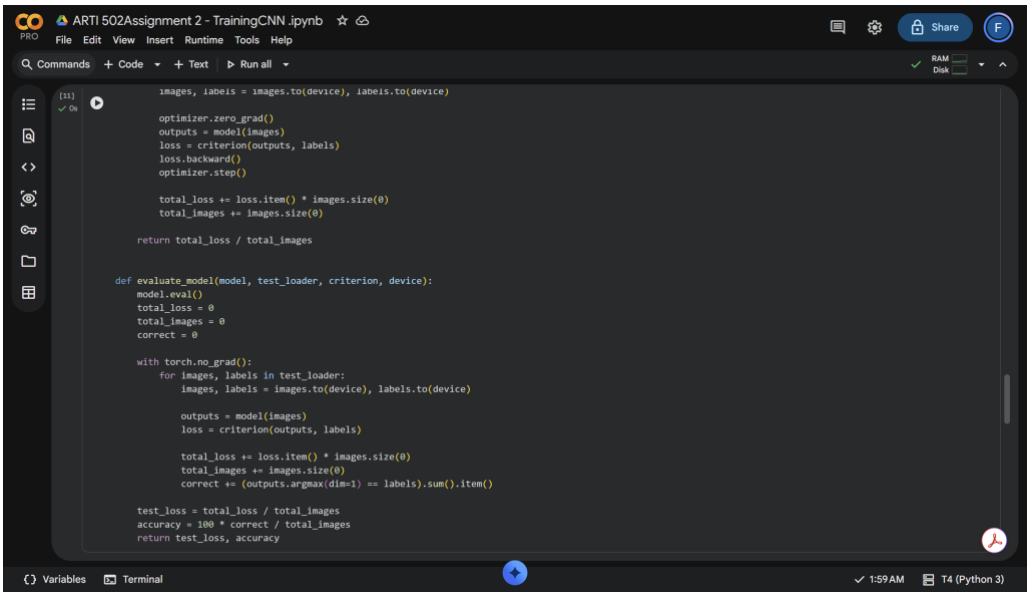


Figure 6: Testing function for the baseline CNN

Public - عام

Public - عام

Task 3:
Loading Data

3.1 Image Tensor Sizes

The dimensions of the image tensors were examined by retrieving one batch from the training DataLoader.

The batch shape was [64, 3, 32, 32], representing 64 images, 3 RGB color channels, and an image size of 32 × 32 pixels. The labels had a shape of [64], indicating one class label for each image.

This confirmed that the images and labels were loaded with the expected dimensions.

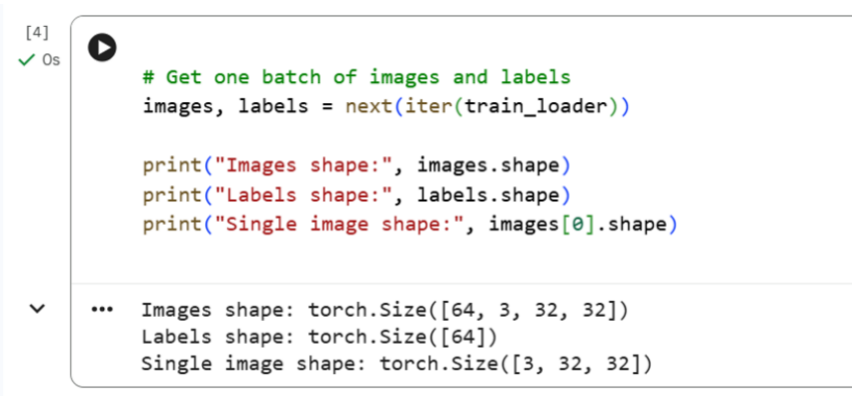


Figure 7: Image Tensor Dimensions and Output

3.2 Dataset Visualization

Matplotlib was used to visualize 10 sample images from the CIFAR-10 training dataset. The images were displayed in a 2 × 5 grid, with each image labeled according to its corresponding class.

The image tensors were rearranged from the PyTorch format (channels, height, width) to the format (height, width, channels) for correct visualization.

The results showed that the images and their class labels were successfully loaded and displayed. The images appeared relatively low-resolution because CIFAR-10 images are only 32 × 32 pixels.

Public - عام

Public - عام

[5] ✓ 0s

```python
# CIFAR-10 class names
classes = train_dataset.classes

# Display 10 sample images
fig, axes = plt.subplots(2, 5, figsize=(12, 5))

for i, ax in enumerate(axes.flat):
    image = images[i].permute(1, 2, 0)
    ax.imshow(image)
    ax.set_title(classes[labels[i].item()])
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Figure 8: CIFAR-10 Dataset Visualization Code



Figure 9: Visualization of 10 CIFAR-10 Sample Images with Class Labels

Task 4: Dataset Augmentation

4.1 Augmentation Implementation

We defined six settings to compare training without augmentation against different combinations of horizontal flipping, resized cropping, rotation, color jitter, and Gaussian noise. The training images are converted to tensors and normalized after the selected augmentations. The test images are converted to tensors and normalized using the same values, without random augmentation.

Public - عام

Public - عام


```python
class GaussianNoise:
    def __init__(self, std=0.03):
        self.std = std

    def __call__(self, tensor):
        return (tensor + torch.randn_like(tensor) * self.std).clamp(0, 1)

augmentation_configs = {
    "Baseline (No Augmentation)": [],
    "Horizontal Flip": ["flip"],
    "Resized Crop + Flip": ["crop", "flip"],
    "Flip + Rotation": ["flip", "rotation"],
    "Color Jitter + Noise": ["color", "noise"],
    "All Augmentations": ["crop", "flip", "rotation", "color", "noise"],
}
```

```python
def make_train_transform(selected):
    steps = []

    if "crop" in selected:
        steps.append(
            transforms.RandomResizedCrop(
                32, scale=(0.85, 1.0), ratio=(0.9, 1.1)
            )
        )
    if "flip" in selected:
        steps.append(transforms.RandomHorizontalFlip(p=0.5))
    if "rotation" in selected:
        steps.append(transforms.RandomRotation(10))
    if "color" in selected:
        steps.append(
            transforms.ColorJitter(0.2, 0.2, 0.2, 0.03)
        )

    steps.append(transforms.ToTensor())

    if "noise" in selected:
        steps.append(GaussianNoise(0.03))

    steps.append(transforms.Normalize(CIFAR_MEAN, CIFAR_STD))
    return transforms.Compose(steps)

test_transform_aug = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(CIFAR_MEAN, CIFAR_STD)
])

print("Ready:", len(augmentation_configs), "augmentation settings")
```

```
Ready: 6 augmentation settings
```

Figure 10: Implementation of six data augmentation configurations and image transformation functions for CIFAR-10.

4.2 Original and Augmented Images

Figure 4.1 shows one original CIFAR-10 image alongside examples of horizontal flipping, resized cropping, rotation, color jitter, Gaussian noise, and a combination of these augmentations. The images are naturally pixelated because CIFAR-10 images are only 32 × 32 pixels.

Public - عام

Public - عام

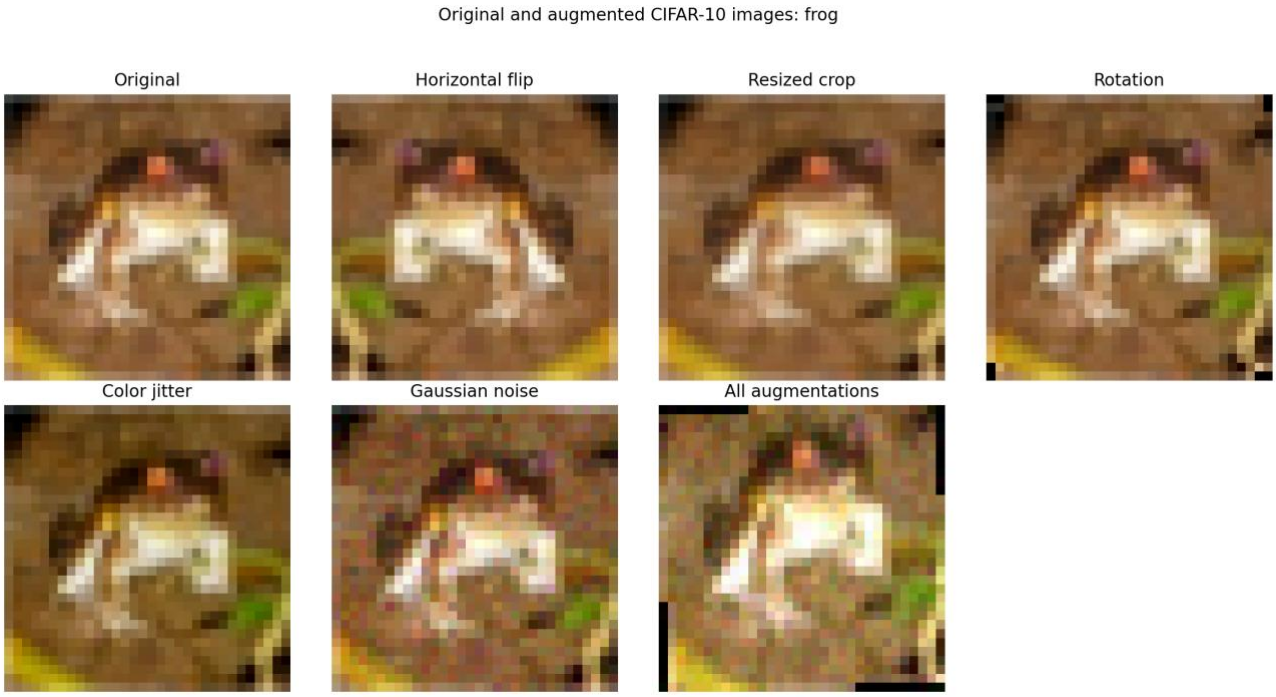


Figure 11: Original CIFAR-10 frog image compared with individual and combined augmentation techniques.

Public - عام

Public - عام

Task 5:

Training Function and Tuning hyperparameters

5.1 Baseline Training Results:

We trained the baseline CNN for five epochs using a batch size of 64 and the Adam optimizer with a learning rate of 0.001. The training and test losses decreased over the epochs, while the test accuracy improved. In the final epoch, the model achieved a training loss of 0.7547, a test loss of 0.8623, and a test accuracy of 69.88%.

Epoch	Training Loss	Test Loss	Test Accuracy
1	1.4586	1.2011	56.64%
2	1.0842	0.9899	65.28%
3	0.9300	0.9234	67.78%
4	0.8365	0.9183	67.92%
5	0.7547	0.8623	69.88%

Figure 12: Baseline CNN training and testing results over five epochs

Public - عام

Public - عام

5.2
Augmentation Experiments

We trained the same CNN for five epochs under six augmentation settings, using a batch size of 64 and the Adam optimizer with a learning rate of 0.001. All settings used the same normalization and test data. Horizontal flipping achieved the best result: a training loss of 0.6347, a test loss of 0.7728,

and a test accuracy of 73.75%. Compared with the no-augmentation setting (71.42%), this improved test accuracy by 2.33 percentage points. Combining all augmentations achieved 70.56%, showing that adding more transformations did not improve performance in this experiment

```
for config_name, selected in augmentation_configs.items():
    random.seed(AUG_SEED)
    np.random.seed(AUG_SEED)
    torch.manual_seed(AUG_SEED)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(AUG_SEED)

    train_dataset_aug = torchvision.datasets.CIFAR10(
        root=DATA_PATH,
        train=True,
        download=False,
        transform=make_train_transform(selected)
    )

    train_loader_aug = DataLoader(
        train_dataset_aug,
        batch_size=AUG_BATCH_SIZE,
        shuffle=True,
        generator=torch.Generator().manual_seed(AUG_SEED)
    )

    experiment_model = BaselineCNN().to(device)
    experiment_criterion = nn.CrossEntropyLoss()
    experiment_optimizer = optim.Adam(
        experiment_model.parameters(), lr=0.001
    )

    experiment_history = {
        "train_loss": [],
        "test_loss": [],
        "test_accuracy": []
    }

    print(f"\n=== {config_name} ===")

    for epoch in range(AUG_EPOCHS):
        train_loss = train_one_epoch(
            experiment_model,
            train_loader_aug,
            experiment_criterion,
            experiment_optimizer,
            device
        )

        test_loss, test_accuracy = evaluate_model(
            experiment_model,
            test_loader_aug,
            experiment_criterion,
            device
        )

        experiment_history["train_loss"].append(train_loss)
        experiment_history["test_loss"].append(test_loss)
        experiment_history["test_accuracy"].append(test_accuracy)

        print(
            f"Epoch {epoch + 1}/{AUG_EPOCHS} | "
            f"Train loss: {train_loss:.4f} | "
            f"Test loss: {test_loss:.4f} | "
            f"Test accuracy: {test_accuracy:.2f}%"
        )

    augmentation_histories[config_name] = experiment_history

print("\nAll augmentation experiments completed.")
```

Figure 13: CNN Training and Evaluation Code

	Augmentation	Train Loss	Test Loss	Test Accuracy (%)
0	Horizontal Flip	0.6347	0.7728	73.75
1	Flip + Rotation	0.7633	0.7889	73.05
2	Resized Crop + Flip	0.7644	0.8032	72.37
3	Baseline (No Augmentation)	0.5448	0.8696	71.42
4	All Augmentations	0.9348	0.8687	70.56
5	Color Jitter + Noise	0.6305	0.8940	70.26

```
Best setting: Horizontal Flip
Best test accuracy: 73.75%
Difference from no augmentation: +2.33 percentage points
```

Figure 14: Data Augmentation Performance Comparison

Public - عام

Public - عام

5.3 CNN Architecture Experiments

Three alternative Convolutional Neural Network (CNN) architectures were implemented and evaluated using the CIFAR-10 dataset. The objective was to investigate how increasing the number of convolutional layers and filters affects image classification performance.

All architectures were trained for five epochs using the Adam optimizer, a learning rate of 0.001, and a batch size of 64 to ensure a consistent comparison.

5.3.1 Experimental Setup

The experiments were conducted using PyTorch with GPU acceleration. The same CIFAR-10 training and testing datasets, loss function (CrossEntropyLoss), and training settings were used for all three architectures.

Each model was evaluated using training loss, test loss, and test accuracy.

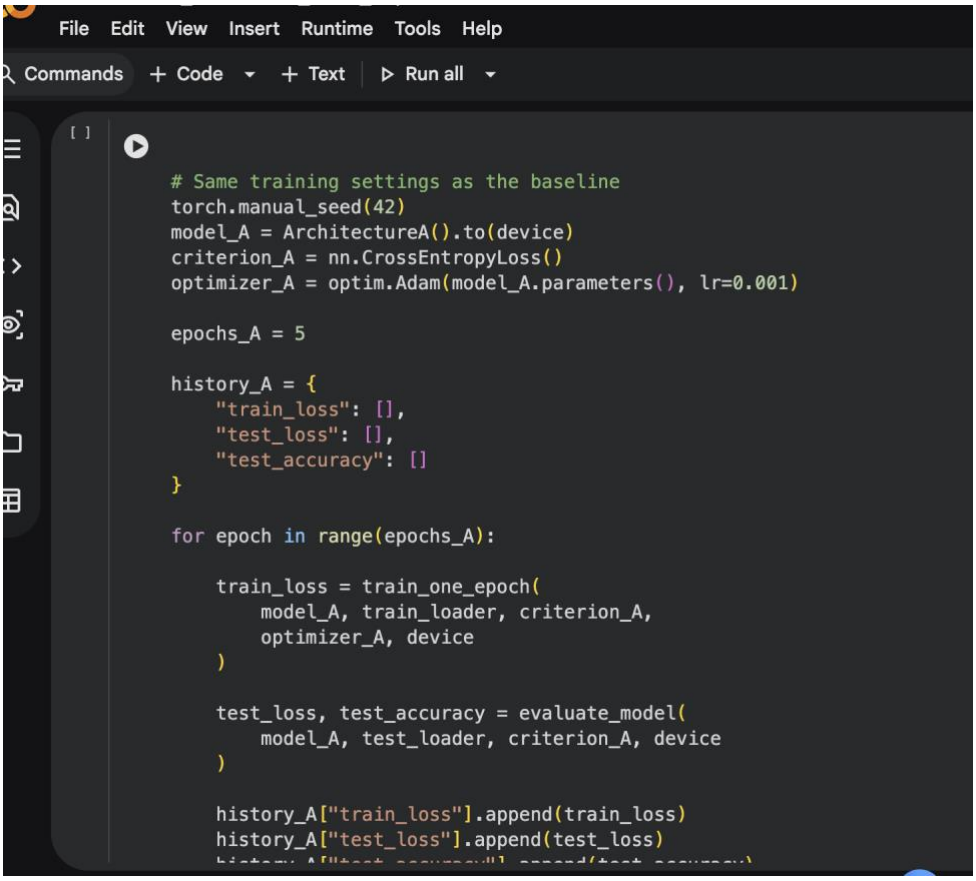


Figure 15: Training Configuration for CNN Architecture Experiments.

Public - عام

Public - عام

5.3.2 Architecture A – More Filters CNN

Architecture A consists of two convolutional layers with 64 and 128 filters. ReLU activation and max-pooling were applied after each convolutional layer.

This architecture was designed to evaluate whether increasing the number of filters improves classification performance without increasing network depth.

After five epochs, Architecture A achieved a test accuracy of **70.94%** and a test loss of **0.8646**.

```python
# Architecture A: More Filters
# ========================================

import torch
import torch.nn as nn

class ArchitectureA(nn.Module):
    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 8 * 8, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.features(x)
```

Figure 16: Architecture A CNN Model Implementation.

```python
    train_loss = train_one_epoch(
        model_A, train_loader, criterion_A,
        optimizer_A, device
    )

    test_loss, test_accuracy = evaluate_model(
        model_A, test_loader, criterion_A, device
    )

    history_A["train_loss"].append(train_loss)
    history_A["test_loss"].append(test_loss)
    history_A["test_accuracy"].append(test_accuracy)

    print(
        f"Epoch [{epoch+1}/{epochs_A}] | "
        f"Train Loss: {train_loss:.4f} | "
        f"Test Loss: {test_loss:.4f} | "
        f"Test Accuracy: {test_accuracy:.2f}%"
    )

print("\nArchitecture A Training Completed!")
```

```
Epoch [1/5] | Train Loss: 1.4018 | Test Loss: 1.1522 | Test Accuracy: 59.25%
Epoch [2/5] | Train Loss: 1.0301 | Test Loss: 0.9337 | Test Accuracy: 67.24%
Epoch [3/5] | Train Loss: 0.8776 | Test Loss: 0.9184 | Test Accuracy: 68.58%
Epoch [4/5] | Train Loss: 0.7778 | Test Loss: 0.8784 | Test Accuracy: 69.30%
Epoch [5/5] | Train Loss: 0.6900 | Test Loss: 0.8646 | Test Accuracy: 70.94%

Architecture A Training Completed!
```

Figure 17: Architecture A Training and Testing Results.

Public - عام

Public - عام

5.3.3
Architecture B – Three-Layer CNN

Architecture B consists of three convolutional layers with 32, 64, and 128 filters. Each convolutional layer is followed by ReLU activation and max-pooling.

The additional convolutional layer allows the model to learn more complex image features.

After five epochs, Architecture B achieved a test accuracy of **72.00%** and a test loss of **0.8154**.

```python
import torch.nn as nn

class ArchitectureB(nn.Module):
    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 4 * 4, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```

Figure 18: Architecture B CNN Model Implementation.

```
[ ]
    train_loss = train_one_epoch(
        model_B, train_loader, criterion_B,
        optimizer_B, device
    )

    test_loss, test_accuracy = evaluate_model(
        model_B, test_loader, criterion_B, device
    )

    history_B["train_loss"].append(train_loss)
    history_B["test_loss"].append(test_loss)
    history_B["test_accuracy"].append(test_accuracy)

    print(
        f"Epoch [{epoch+1}/{epochs_B}] | "
        f"Train Loss: {train_loss:.4f} | "
        f"Test Loss: {test_loss:.4f} | "
        f"Test Accuracy: {test_accuracy:.2f}%"
    )

print("\nArchitecture B Training Completed!")

...  Epoch [1/5] | Train Loss: 1.5529 | Test Loss: 1.2821 | Test Accuracy: 53.28%
     Epoch [2/5] | Train Loss: 1.1432 | Test Loss: 1.0366 | Test Accuracy: 63.15%
     Epoch [3/5] | Train Loss: 0.9438 | Test Loss: 0.9102 | Test Accuracy: 68.36%
     Epoch [4/5] | Train Loss: 0.8159 | Test Loss: 0.8294 | Test Accuracy: 71.53%
     Epoch [5/5] | Train Loss: 0.7255 | Test Loss: 0.8154 | Test Accuracy: 72.00%

     Architecture B Training Completed!
```

Figure 19: Architecture B Training and Testing Results.

Public - عام

Public - عام

5.3.4
Architecture C – Four-Layer CNN

Architecture C extends the network to four convolutional layers with 32, 64, 128, and 256 filters. Each convolutional layer is followed by ReLU activation and max-pooling.

This architecture was designed to investigate whether increasing network depth further improves classification accuracy.

After five epochs, Architecture C achieved a test accuracy of **72.69%** and a test loss of **0.8029**.

```python
#  Architecture C: Deeper CNN
# Four Convolutional Layers
# ======================================

import torch
import torch.nn as nn

class ArchitectureC(nn.Module):
    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256 * 2 * 2, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```

Figure 20: Architecture C CNN Model Implementation.

```python
        model_C, train_loader, criterion_C,
        optimizer_C, device
    )

    test_loss, test_accuracy = evaluate_model(
        model_C, test_loader, criterion_C, device
    )

    history_C["train_loss"].append(train_loss)
    history_C["test_loss"].append(test_loss)
    history_C["test_accuracy"].append(test_accuracy)

    print(
        f"Epoch [{epoch+1}/{epochs_C}] | "
        f"Train Loss: {train_loss:.4f} | "
        f"Test Loss: {test_loss:.4f} | "
        f"Test Accuracy: {test_accuracy:.2f}%"
    )

print("\nArchitecture C Training Completed!")
```

```
Epoch [1/5] | Train Loss: 1.5889 | Test Loss: 1.2903 | Test Accuracy: 52.70%
Epoch [2/5] | Train Loss: 1.1175 | Test Loss: 1.0786 | Test Accuracy: 61.70%
Epoch [3/5] | Train Loss: 0.8997 | Test Loss: 0.8668 | Test Accuracy: 69.33%
Epoch [4/5] | Train Loss: 0.7636 | Test Loss: 0.8450 | Test Accuracy: 71.16%
Epoch [5/5] | Train Loss: 0.6551 | Test Loss: 0.8029 | Test Accuracy: 72.69%

Architecture C Training Completed!
```

Figure 21: Architecture C Training and Testing Results.

Public - عام

Public - عام

5.3.5 Results Comparison

The three CNN architectures were compared with the baseline model using training loss, test loss, and test accuracy.

Architecture	Conv Layers	Filters	Train Loss	Test Loss	Test Accuracy
Baseline	2	32, 64	0.7976	0.9002	69.10%
Architecture A	2	64, 128	0.6900	0.8646	70.94%
Architecture B	3	32, 64, 128	0.7255	0.8154	72.00%
Architecture C	4	32, 64, 128, 256	0.6551	0.8029	72.69%

The results indicate that increasing the number of convolutional layers improved test accuracy under the evaluated settings. Architecture C achieved the highest test accuracy and the lowest test loss.

The following graphs illustrate the performance of the three architectures across five epochs.

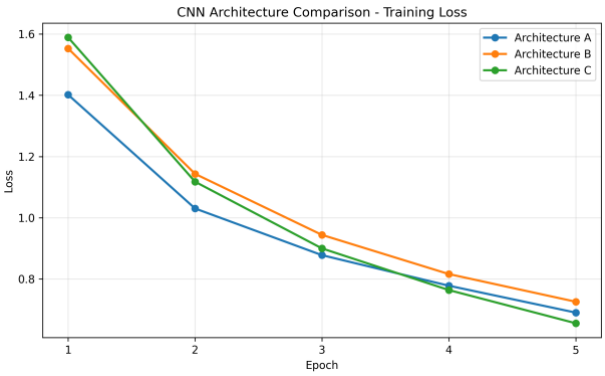


Figure 22: Training Loss Comparison Across CNN Architectures

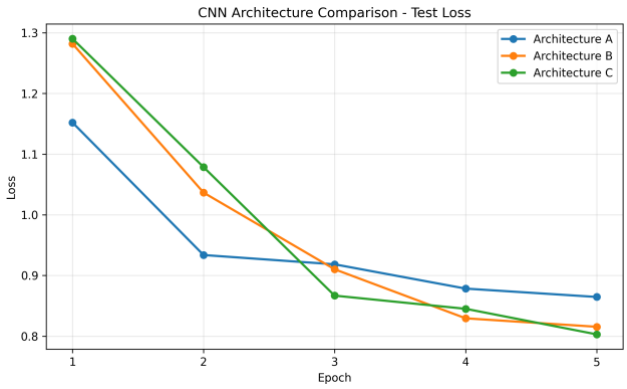


Figure 23: Test Loss Comparison Across CNN Architectures.

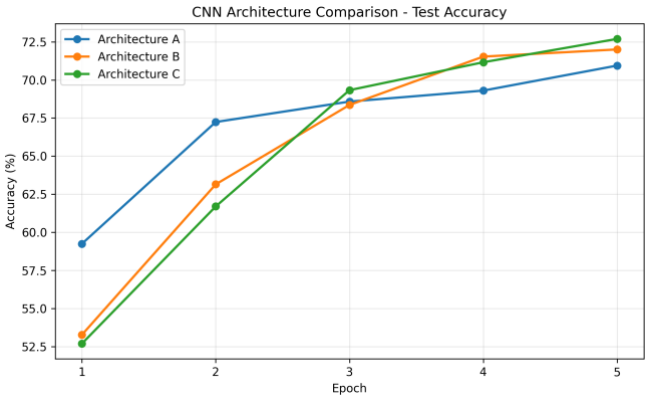


Figure 24: Test Accuracy Comparison Across CNN Architectures.

Public - عام

Public - عام

5.3.6 Best Architecture Selection

Based on the experimental results, Architecture C was selected as the best-performing CNN architecture.

It achieved a test accuracy of 72.69%, representing an improvement of 3.59 percentage points compared with the baseline CNN (69.10%).

Architecture C also achieved the lowest test loss of 0.8029. However, its accuracy improvement over Architecture B was only 0.69 percentage points, indicating that the additional convolutional layer provided a relatively small improvement.

Overall, Architecture C demonstrated the best performance among the evaluated architectures.

5.4 Hyperparameter Experiments

5.4.1 This function trains and evaluates the Baseline CNN using different hyperparameter settings. It records the training loss, test loss, and test accuracy for each experiment.

Hyperparameter Experiments

```
from torch.utils.data import DataLoader
import pandas as pd

def run_tuning_experiment(lr, batch_size, epochs, experiment_name):

    train_loader_exp = DataLoader(
        train_dataset,
        batch_size=batch_size,
        shuffle=True
    )

    test_loader_exp = DataLoader(
        test_dataset,
        batch_size=batch_size,
        shuffle=False
    )

    model = BaselineCNN().to(device)

    criterion = nn.CrossEntropyLoss()

    optimizer = optim.Adam(
```

Figure 25: Hyperparameter Tuning Function Implementation

Public - عام

Public - عام

5.4.2 Learning Rate Experiments

This experiment tests three different learning rates (0.01, 0.001, and 0.0001) while keeping the batch size and number of epochs constant.

```
lr_results = []

learning_rates = [0.01, 0.001, 0.0001]

for lr in learning_rates:

    result = run_tuning_experiment(
        lr=lr,
        batch_size=64,
        epochs=10,
        experiment_name=f"Learning Rate {lr}"
    )

    lr_results.append(result)
```

Figure 26: Learning Rate Experiment Implementation

```
Experiment: Learning Rate 0.01
Epoch 1/10 | Train Loss: 1.5809 | Test Loss: 1.4446 | Accuracy: 47.74%
Epoch 2/10 | Train Loss: 1.3479 | Test Loss: 1.2832 | Accuracy: 53.92%
Epoch 3/10 | Train Loss: 1.2817 | Test Loss: 1.3006 | Accuracy: 53.35%
Epoch 4/10 | Train Loss: 1.2363 | Test Loss: 1.2623 | Accuracy: 55.15%
Epoch 5/10 | Train Loss: 1.1963 | Test Loss: 1.2997 | Accuracy: 54.35%
Epoch 6/10 | Train Loss: 1.1752 | Test Loss: 1.3051 | Accuracy: 54.88%
Epoch 7/10 | Train Loss: 1.1411 | Test Loss: 1.2688 | Accuracy: 56.38%
Epoch 8/10 | Train Loss: 1.1185 | Test Loss: 1.2899 | Accuracy: 56.46%
Epoch 9/10 | Train Loss: 1.1048 | Test Loss: 1.2641 | Accuracy: 55.86%
Epoch 10/10 | Train Loss: 1.0876 | Test Loss: 1.3180 | Accuracy: 55.41%

Experiment: Learning Rate 0.001
Epoch 1/10 | Train Loss: 1.4998 | Test Loss: 1.2511 | Accuracy: 56.18%
Epoch 2/10 | Train Loss: 1.1288 | Test Loss: 1.0783 | Accuracy: 61.42%
Epoch 3/10 | Train Loss: 0.9825 | Test Loss: 0.9523 | Accuracy: 66.77%
Epoch 4/10 | Train Loss: 0.8779 | Test Loss: 0.9132 | Accuracy: 68.00%
Epoch 5/10 | Train Loss: 0.8050 | Test Loss: 0.8939 | Accuracy: 69.22%
Epoch 6/10 | Train Loss: 0.7445 | Test Loss: 0.9282 | Accuracy: 67.61%
Epoch 7/10 | Train Loss: 0.6832 | Test Loss: 0.8772 | Accuracy: 69.95%
Epoch 8/10 | Train Loss: 0.6277 | Test Loss: 0.8857 | Accuracy: 69.89%
Epoch 9/10 | Train Loss: 0.5789 | Test Loss: 0.8750 | Accuracy: 70.78%
Epoch 10/10 | Train Loss: 0.5242 | Test Loss: 0.9088 | Accuracy: 70.29%
```

```
Experiment: Learning Rate 0.0001
Epoch 1/10 | Train Loss: 1.8176 | Test Loss: 1.5884 | Accuracy: 43.41%
Epoch 2/10 | Train Loss: 1.5371 | Test Loss: 1.4713 | Accuracy: 48.26%
Epoch 3/10 | Train Loss: 1.4240 | Test Loss: 1.3674 | Accuracy: 51.48%
Epoch 4/10 | Train Loss: 1.3444 | Test Loss: 1.3335 | Accuracy: 53.51%
Epoch 5/10 | Train Loss: 1.2864 | Test Loss: 1.2627 | Accuracy: 55.12%
Epoch 6/10 | Train Loss: 1.2357 | Test Loss: 1.2367 | Accuracy: 56.12%
Epoch 7/10 | Train Loss: 1.1984 | Test Loss: 1.2032 | Accuracy: 57.62%
Epoch 8/10 | Train Loss: 1.1615 | Test Loss: 1.1612 | Accuracy: 58.79%
Epoch 9/10 | Train Loss: 1.1301 | Test Loss: 1.1434 | Accuracy: 59.90%
Epoch 10/10 | Train Loss: 1.0997 | Test Loss: 1.1099 | Accuracy: 61.48%
```

Figure 27: CNN Training and Evaluation Results for Different Learning Rates

Public - عام

Public - عام

5.4.3 Batch Size Experiments

This experiment compares three batch sizes (32, 64, and 128) while keeping the learning rate and number of epochs constant.

```python
batch_results = []

batch_sizes = [32, 64, 128]

for batch in batch_sizes:

    result = run_tuning_experiment(
        lr=0.001,
        batch_size=batch,
        epochs=10,
        experiment_name=f"Batch Size {batch}"
    )

    batch_results.append(result)
```

Figure 28: Batch Size Experiment Implementation

```
...
    Experiment: Batch Size 32
    Epoch 1/10 | Train Loss: 1.4087 | Test Loss: 1.1147 | Accuracy: 60.64%
    Epoch 2/10 | Train Loss: 1.0366 | Test Loss: 0.9691 | Accuracy: 65.77%
    Epoch 3/10 | Train Loss: 0.8900 | Test Loss: 0.9329 | Accuracy: 67.56%
    Epoch 4/10 | Train Loss: 0.7926 | Test Loss: 0.8959 | Accuracy: 68.95%
    Epoch 5/10 | Train Loss: 0.7039 | Test Loss: 0.9150 | Accuracy: 68.92%
    Epoch 6/10 | Train Loss: 0.6286 | Test Loss: 0.8720 | Accuracy: 70.80%
    Epoch 7/10 | Train Loss: 0.5557 | Test Loss: 0.9084 | Accuracy: 70.46%
    Epoch 8/10 | Train Loss: 0.4884 | Test Loss: 0.9283 | Accuracy: 70.56%
    Epoch 9/10 | Train Loss: 0.4263 | Test Loss: 0.9994 | Accuracy: 69.98%
    Epoch 10/10 | Train Loss: 0.3695 | Test Loss: 1.0672 | Accuracy: 70.50%

    Experiment: Batch Size 64
    Epoch 1/10 | Train Loss: 1.4743 | Test Loss: 1.2219 | Accuracy: 55.90%
    Epoch 2/10 | Train Loss: 1.1045 | Test Loss: 1.0206 | Accuracy: 64.18%
    Epoch 3/10 | Train Loss: 0.9460 | Test Loss: 0.9488 | Accuracy: 66.42%
    Epoch 4/10 | Train Loss: 0.8448 | Test Loss: 0.8929 | Accuracy: 68.90%
    Epoch 5/10 | Train Loss: 0.7673 | Test Loss: 0.8914 | Accuracy: 69.20%
    Epoch 6/10 | Train Loss: 0.6955 | Test Loss: 0.8352 | Accuracy: 71.49%
    Epoch 7/10 | Train Loss: 0.6320 | Test Loss: 0.8748 | Accuracy: 70.49%
    Epoch 8/10 | Train Loss: 0.5715 | Test Loss: 0.9307 | Accuracy: 69.25%
    Epoch 9/10 | Train Loss: 0.5103 | Test Loss: 0.8839 | Accuracy: 71.31%
    Epoch 10/10 | Train Loss: 0.4549 | Test Loss: 0.9242 | Accuracy: 71.07%
```

```
    Experiment: Batch Size 128
    Epoch 1/10 | Train Loss: 1.5447 | Test Loss: 1.2678 | Accuracy: 54.53%
    Epoch 2/10 | Train Loss: 1.2035 | Test Loss: 1.1372 | Accuracy: 59.49%
    Epoch 3/10 | Train Loss: 1.0467 | Test Loss: 1.0446 | Accuracy: 63.20%
    Epoch 4/10 | Train Loss: 0.9461 | Test Loss: 0.9720 | Accuracy: 65.94%
    Epoch 5/10 | Train Loss: 0.8555 | Test Loss: 0.9089 | Accuracy: 68.26%
    Epoch 6/10 | Train Loss: 0.7835 | Test Loss: 0.9035 | Accuracy: 68.26%
    Epoch 7/10 | Train Loss: 0.7237 | Test Loss: 0.8716 | Accuracy: 69.70%
    Epoch 8/10 | Train Loss: 0.6665 | Test Loss: 0.8443 | Accuracy: 70.73%
    Epoch 9/10 | Train Loss: 0.6223 | Test Loss: 0.8649 | Accuracy: 70.71%
    Epoch 10/10 | Train Loss: 0.5695 | Test Loss: 0.8714 | Accuracy: 71.06%
```

Figure 29: CNN Training and Evaluation Results for Different Batch Sizes

Public - عام

Public - عام

5.4.4 Epochs Experiments

This experiment tests different numbers of epochs (5, 10, and 15) while keeping the learning rate and batch size constant

```python
epoch_results = []

epoch_values = [5, 10, 15]

for ep in epoch_values:

    result = run_tuning_experiment(
        lr=0.001,
        batch_size=64,
        epochs=ep,
        experiment_name=f"Epochs {ep}"
    )

    epoch_results.append(result)
```

Figure 30: Epoch Experiment Implementation

```
Experiment: Epochs 5
Epoch 1/5 | Train Loss: 1.5111 | Test Loss: 1.2406 | Accuracy: 56.51%
Epoch 2/5 | Train Loss: 1.1552 | Test Loss: 1.0661 | Accuracy: 62.08%
Epoch 3/5 | Train Loss: 0.9918 | Test Loss: 1.0717 | Accuracy: 62.04%
Epoch 4/5 | Train Loss: 0.8845 | Test Loss: 0.9040 | Accuracy: 68.53%
Epoch 5/5 | Train Loss: 0.8001 | Test Loss: 0.8855 | Accuracy: 69.10%

Experiment: Epochs 10
Epoch 1/10 | Train Loss: 1.5058 | Test Loss: 1.2737 | Accuracy: 54.67%
Epoch 2/10 | Train Loss: 1.1487 | Test Loss: 1.0651 | Accuracy: 62.09%
Epoch 3/10 | Train Loss: 0.9930 | Test Loss: 1.0192 | Accuracy: 64.25%
Epoch 4/10 | Train Loss: 0.8935 | Test Loss: 0.9224 | Accuracy: 67.67%
Epoch 5/10 | Train Loss: 0.8146 | Test Loss: 0.9158 | Accuracy: 67.86%
Epoch 6/10 | Train Loss: 0.7514 | Test Loss: 0.8817 | Accuracy: 69.33%
Epoch 7/10 | Train Loss: 0.6909 | Test Loss: 0.8821 | Accuracy: 69.67%
Epoch 8/10 | Train Loss: 0.6334 | Test Loss: 0.8751 | Accuracy: 70.65%
Epoch 9/10 | Train Loss: 0.5789 | Test Loss: 0.8844 | Accuracy: 70.60%
Epoch 10/10 | Train Loss: 0.5246 | Test Loss: 0.9226 | Accuracy: 70.55%
```

```
Experiment: Epochs 15
Epoch 1/15 | Train Loss: 1.4986 | Test Loss: 1.2508 | Accuracy: 55.24%
Epoch 2/15 | Train Loss: 1.1341 | Test Loss: 1.0670 | Accuracy: 62.26%
Epoch 3/15 | Train Loss: 0.9744 | Test Loss: 0.9637 | Accuracy: 66.53%
Epoch 4/15 | Train Loss: 0.8844 | Test Loss: 0.9415 | Accuracy: 67.42%
Epoch 5/15 | Train Loss: 0.8133 | Test Loss: 0.9194 | Accuracy: 67.93%
Epoch 6/15 | Train Loss: 0.7548 | Test Loss: 0.8903 | Accuracy: 69.69%
Epoch 7/15 | Train Loss: 0.6952 | Test Loss: 0.8857 | Accuracy: 69.91%
Epoch 8/15 | Train Loss: 0.6442 | Test Loss: 0.8927 | Accuracy: 69.79%
Epoch 9/15 | Train Loss: 0.5943 | Test Loss: 0.9240 | Accuracy: 69.40%
Epoch 10/15 | Train Loss: 0.5450 | Test Loss: 0.9298 | Accuracy: 69.91%
Epoch 11/15 | Train Loss: 0.5073 | Test Loss: 0.9212 | Accuracy: 69.90%
Epoch 12/15 | Train Loss: 0.4547 | Test Loss: 1.0096 | Accuracy: 70.23%
Epoch 13/15 | Train Loss: 0.4152 | Test Loss: 0.9928 | Accuracy: 69.89%
Epoch 14/15 | Train Loss: 0.3777 | Test Loss: 1.0911 | Accuracy: 69.34%
Epoch 15/15 | Train Loss: 0.3369 | Test Loss: 1.1910 | Accuracy: 68.37%
```

Figure 31: CNN Training and Evaluation Results for Different Numbers of Epochs

Public - عام

Public - عام

5.4.5
Hyperparameter Tuning Summary

```
26]
✓ 0s
    all_results = (
        lr_results +
        batch_results +
        epoch_results
    )

    results_df = pd.DataFrame(all_results)

    print(results_df.to_string(index=False))
```

Experiment	Learning Rate	Batch Size	Epochs	Training Loss	Test Loss	Test Accuracy
Learning Rate 0.01	0.0100	64	10	1.087573	1.318011	55.41
Learning Rate 0.001	0.0010	64	10	0.524180	0.908772	70.29
Learning Rate 0.0001	0.0001	64	10	1.099725	1.109931	61.48
Batch Size 32	0.0010	32	10	0.369488	1.067210	70.50
Batch Size 64	0.0010	64	10	0.454893	0.924236	71.07
Batch Size 128	0.0010	128	10	0.569540	0.871363	71.06
Epochs 5	0.0010	64	5	0.800109	0.885496	69.10
Epochs 10	0.0010	64	10	0.524595	0.922586	70.55
Epochs 15	0.0010	64	15	0.336890	1.190966	68.37

Figure 32: Summary of Hyperparameter Tuning Results

Public - عام

Public - عام

5.5 Final Model Training and Testing

5.5.1 Final Model Configuration

Based on the previous experiments, the Deeper CNN architecture was selected as the final model because it achieved the best performance among the tested architectures. Horizontal flipping was also selected as the augmentation technique because it provided the highest accuracy in the augmentation experiments.

The final model was trained using the best hyperparameters selected from the previous experiments: the Adam optimizer, a learning rate of 0.001, a batch size of 128, and 10 epochs. GPU acceleration (CUDA) was used to speed up the training process.

```python
# Code 1: Final Model Configuration

import torch
import torchvision
import torch.nn as nn
import torch.optim as optim
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import json
from torch.utils.data import DataLoader

FINAL_SEED = 42

FINAL_LR = float(best_lr["Learning Rate"])
FINAL_BATCH_SIZE = int(best_batch["Batch Size"])
FINAL_EPOCHS = int(best_epochs["Epochs"])

random.seed(FINAL_SEED)
np.random.seed(FINAL_SEED)
torch.manual_seed(FINAL_SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(FINAL_SEED)

# Final training dataset with Horizontal Flip
final_train_dataset = torchvision.datasets.CIFAR10(
    root=DATA_PATH,
    train=True,
    download=True,
    transform=make_train_transform(["flip"])
)

# Test dataset without random augmentation
final_test_dataset = torchvision.datasets.CIFAR10(
    root=DATA_PATH,
    train=False,
```

Figure 33: Final CNN model configuration and selected training parameters.

Public - عام

Public - عام

5.5.2 Final Model Training and Testing Results

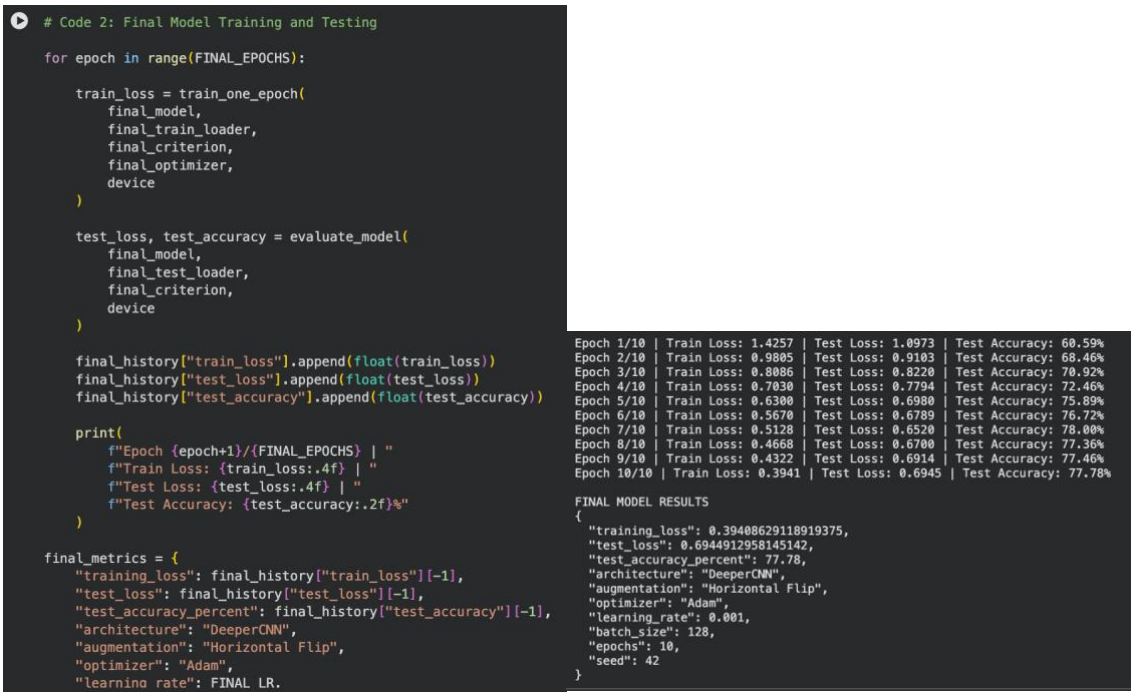
The final CNN model was trained and tested on the CIFAR-10 dataset for 10 epochs. During each epoch, the training loss, test loss, and test accuracy were recorded to evaluate the model's performance.

The training loss decreased from 1.4237 in the first epoch to 0.3941 in the final epoch. The test loss also decreased from 1.0973 to 0.6945, while the test accuracy increased from 60.59% to 77.78%.

The final results were:

- **Training Loss:** 0.3941
- **Test Loss:** 0.6945
- **Test Accuracy:** 77.78%

These results indicate that the final CNN model successfully learned image features and achieved improved classification performance on the CIFAR-10 dataset.



```
# Code 2: Final Model Training and Testing

for epoch in range(FINAL_EPOCHS):

    train_loss = train_one_epoch(
        final_model,
        final_train_loader,
        final_criterion,
        final_optimizer,
        device
    )

    test_loss, test_accuracy = evaluate_model(
        final_model,
        final_test_loader,
        final_criterion,
        device
    )

    final_history["train_loss"].append(float(train_loss))
    final_history["test_loss"].append(float(test_loss))
    final_history["test_accuracy"].append(float(test_accuracy))

    print(
        f"Epoch {epoch+1}/{FINAL_EPOCHS} | "
        f"Train Loss: {train_loss:.4f} | "
        f"Test Loss: {test_loss:.4f} | "
        f"Test Accuracy: {test_accuracy:.2f}%"
    )

final_metrics = {
    "training_loss": final_history["train_loss"][-1],
    "test_loss": final_history["test_loss"][-1],
    "test_accuracy_percent": final_history["test_accuracy"][-1],
    "architecture": "DeeperCNN",
    "augmentation": "Horizontal Flip",
    "optimizer": "Adam",
    "learning_rate": FINAL_LR,
```

```
Epoch 1/10 | Train Loss: 1.4257 | Test Loss: 1.0973 | Test Accuracy: 60.59%
Epoch 2/10 | Train Loss: 0.9805 | Test Loss: 0.9103 | Test Accuracy: 68.46%
Epoch 3/10 | Train Loss: 0.8086 | Test Loss: 0.8220 | Test Accuracy: 70.92%
Epoch 4/10 | Train Loss: 0.7030 | Test Loss: 0.7794 | Test Accuracy: 72.46%
Epoch 5/10 | Train Loss: 0.6300 | Test Loss: 0.6980 | Test Accuracy: 75.89%
Epoch 6/10 | Train Loss: 0.5670 | Test Loss: 0.6789 | Test Accuracy: 76.72%
Epoch 7/10 | Train Loss: 0.5128 | Test Loss: 0.6520 | Test Accuracy: 78.00%
Epoch 8/10 | Train Loss: 0.4668 | Test Loss: 0.6700 | Test Accuracy: 77.36%
Epoch 9/10 | Train Loss: 0.4322 | Test Loss: 0.6914 | Test Accuracy: 77.46%
Epoch 10/10 | Train Loss: 0.3941 | Test Loss: 0.6945 | Test Accuracy: 77.78%

FINAL MODEL RESULTS
{
  "training_loss": 0.39408629118919375,
  "test_loss": 0.6944912958145142,
  "test_accuracy_percent": 77.78,
  "architecture": "DeeperCNN",
  "augmentation": "Horizontal Flip",
  "optimizer": "Adam",
  "learning_rate": 0.001,
  "batch_size": 128,
  "epochs": 10,
  "seed": 42
}
```

Figure 34: Final model training and testing results over 10 epochs.

Public - عام

Public - عام

5.6 Final Graphs and Performance Analysis

5.6.1 Final Model Performance Graphs

Three performance graphs were generated to visualize the final CNN model's training loss, test loss, and test accuracy over 10 epochs.

Training Loss: The training loss decreased consistently from 1.4237 to 0.3941, indicating that the model learned effectively from the training dataset.

Test Loss: The test loss decreased from 1.0973 to 0.6945. However, a slight increase was observed during the final epochs, suggesting possible overfitting.

Test Accuracy: The test accuracy increased from 60.59% to 77.78%, showing an overall improvement in the model's classification performance.

```python
# Code 3: Final Model Performance Graphs

epochs_axis = range(
    1, len(final_history["train_loss"]) + 1
)

fig, axes = plt.subplots(1, 3, figsize=(16, 4))

# Training Loss
axes[0].plot(
    epochs_axis,
    final_history["train_loss"],
    marker="o"
)
axes[0].set_title("Training Loss")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Loss")
axes[0].grid(True, alpha=0.3)

# Test Loss
axes[1].plot(
    epochs_axis,
    final_history["test_loss"],
    marker="o"
)
axes[1].set_title("Test Loss")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Loss")
axes[1].grid(True, alpha=0.3)

# Test Accuracy
axes[2].plot(
    epochs_axis,
    final_history["test_accuracy"],
    marker="o"
)
axes[2].set_title("Test Accuracy")
axes[2].set_xlabel("Epoch")
```

Final Deeper CNN with Horizontal Flip

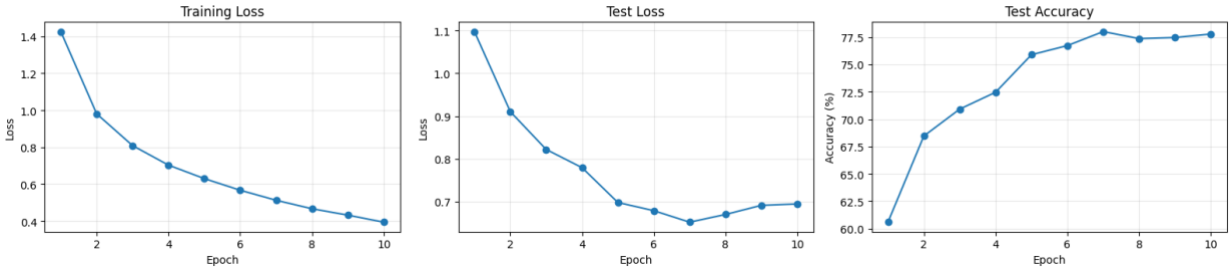


Figure 35: Training loss, test loss, and test accuracy of the final CNN model over 10 epochs.

Public - عام

Public - عام

5.6.2 Final Model vs. Baseline Comparison

The final model was compared with the baseline CNN to evaluate the improvement in classification performance.

The final CNN achieved a test accuracy of 77.78%, compared with 70.52% for the baseline CNN, representing an improvement of 7.26 percentage points.

The final model also achieved lower training and test losses. These results indicate that the selected architecture, augmentation technique, and hyperparameters produced better classification performance than the baseline configuration in this experiment.

```python
# Code 4: Final vs Baseline Comparison

baseline_history = results["Baseline CNN"]

baseline_train_loss = baseline_history["train_loss"][-1]
baseline_test_loss = baseline_history["test_loss"][-1]
baseline_accuracy = baseline_history["test_accuracy"][-1]

comparison_df = pd.DataFrame([
    {
        "Model": "Baseline CNN",
        "Training Loss": baseline_train_loss,
        "Test Loss": baseline_test_loss,
        "Test Accuracy (%)": baseline_accuracy
    },
    {
        "Model": "Final Deeper CNN + Horizontal Flip",
        "Training Loss": final_metrics["training_loss"],
        "Test Loss": final_metrics["test_loss"],
        "Test Accuracy (%)": final_metrics["test_accuracy_percent"]
    }
])

print("\nFinal Model vs Baseline Comparison")
print(comparison_df.to_string(index=False))

accuracy_difference = (
    final_metrics["test_accuracy_percent"]
    - baseline_accuracy
)

print(
    f"\nAccuracy Difference: "
    f"{accuracy_difference:+.2f} percentage points"
)

if accuracy_difference > 0:
    print("Final model achieved higher test accuracy.")
```

```
Final Model vs Baseline Comparison
                          Model  Training Loss  Test Loss  Test Accuracy (%)
                   Baseline CNN       0.596165   0.869138              70.52
Final Deeper CNN + Horizontal Flip       0.394086   0.694491              77.78

Accuracy Difference: +7.26 percentage points
Final model achieved higher test accuracy.
```

Figure 36: Performance comparison between the baseline CNN and the final CNN.

Public - عام

Public - عام

5.6.3 Final Performance Analysis

The experimental results demonstrate that combining the Deeper CNN architecture with horizontal flipping and the selected hyperparameters improved image classification performance on the CIFAR-10 dataset.

The consistent decrease in training loss indicates effective learning, while the improvement in test accuracy shows that the model performed better on unseen images.

Although the test loss increased slightly during the final epochs, the final model maintained a high test accuracy of 77.78%.

Overall, the final CNN outperformed the baseline CNN by 7.26 percentage points in test accuracy, demonstrating the effectiveness of the selected final model configuration.

Task 6: Save your source code

https://github.com/FAshihab/DL-Assignment-2-Group-1-

- End of Assignment -

Public - عام

Public - عام

Grading Rubric

Criterion	Excellent	Good	Developing	Poor	Marks
Task 1: Getting the CIFAR Dataset	Dataset loaded correctly with clear screenshot showing successful execution	Loaded correctly but screenshot is unclear	Dataset loaded with issues or incomplete evidence	Not loaded / missing screenshot	**0.5**
Task 2: Defining the Network	All clear screenshots	Minor mistakes or missing small details	Several missing description or unclear screenshots	Not answered / incorrect / missing evidence	**2**
Task 3: Loading Data	All clear screenshots	Minor mistakes or missing small details	Several missing description or unclear screenshots	Not loaded / missing screenshot	**0.5**
Task 4: Dataset Augmentation	All datasets augmentation implemented	Major datasets augmentation implemented	Minor datasets augmentation implemented	Not done	**2**
Task 5: Training Function and Tuning parameters	Tuning parameters and model architecture created, screenshot included, and resultlts description explained accurately	Tuning Parameters and model architecture correct but results description is weak	Tuning Parameters and model architecture corect but incorrect results description	Not done	**2**
Task 6: Save your source code	Source code save succefully	Source code save succefully done but minor missing item	Source code save incomplete or file format incorrect	Not submitted / missing	**0.5**
Total					**7.5**

Public - عام